Conjugate Gradient Beyond the Quadratic Case

Literature map and extensions after U18–U19 (MMC007 Optimization, IMCA)

Gustavo Lozano

November 2026

Companion material. All paths are relative to the repository root.

- Unit labels (U5, U7, U15–U21, in particular U18 and U19) refer to the optimization agenda `master/IMCA/20262/courses/MMC007/exercises/agenda_optimizacion.pdf`.
- The code and experiments (`linear_cg.py`, `P18.py`, `P19.py`, `P20.py`) are in `master/IMCA/20262/courses/MMC007/exercises/src/`.

Abstract

Units U18–U19 of the course agenda covered linear conjugate gradient (CG) for $\min \frac{1}{2}x^{\mathsf{T}}Ax - b^{\mathsf{T}}x$ with $A \succ 0$. Experiments P18–P20 confirmed two facts: CG terminates in at most as many steps as there are distinct eigenvalues of A, and its rate $2\big(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\big)^k$ is far better than steepest descent's $\big(\frac{\kappa-1}{\kappa+1}\big)^k$. This note answers *whether CG can be used beyond quadratics*. The answer is yes, in three different senses: (i) Krylov relatives of CG solve linear systems that are not symmetric positive definite; (ii) linear CG becomes the *inner solver* of Newton and trust-region methods for general smooth f; (iii) *nonlinear CG* (Fletcher–Reeves, Polak–Ribière, Hager–Zhang, ...) applies the CG recurrence directly to general f. For each sense it states the key results, shows a small experiment, and gives an annotated reading list for when the professor's articles arrive.

Contents

1 Starting point: what P18–P20 established **2**

2 Sense (i): other linear systems (still linear, but not SPD) **2**

3 Sense (ii): linear CG inside Newton and trust-region methods **3**

4 Sense (iii): nonlinear CG for general smooth f **3**
4.1 The algorithm . . . 3
4.2 The choice of β_k . . . 3
4.3 What is known about convergence . . . 4
4.4 Modern implementations and the bridge to L-BFGS (U20) . . . 5
4.5 Further generalizations (research frontier) . . . 5

5 Small experiment **5**

6 Annotated reading list **6**

7 Conclusion **8**

1 Starting point: what P18–P20 established

With $r_k = Ax_k - b = \nabla f(x_k)$, the (preconditioned, $M \succ 0$) linear CG implemented in `linear_cg.py` is

$$\begin{aligned} &\alpha_k = \frac{r_k^\mathsf{T} y_k}{p_k^\mathsf{T} A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k + \alpha_k A p_k, \\ &y_{k+1} = M^{-1} r_{k+1}, \quad \beta_{k+1} = \frac{r_{k+1}^\mathsf{T} y_{k+1}}{r_k^\mathsf{T} y_k}, \quad p_{k+1} = -y_{k+1} + \beta_{k+1} p_k. \end{aligned} \tag{1}$$

The three experiments verified:

- **P18** ($n = 2$): CG reaches x^* in 2 steps. Steepest descent and coordinate descent zig-zag.
- **P19** ($A = \mathrm{diag}(1, 1, 10, 10)$, $n = 4$): CG terminates in 2 steps, the number of *distinct* eigenvalues of A, not n.
- **P20** ($A = \mathrm{diag}(1, 12, \ldots, 100)$, $\kappa = 100$): CG terminates at $k = 10$ because A has 10 distinct eigenvalues. Before that, it follows (and beats) the bound $2(9/11)^k$. Steepest descent follows $(99/101)^k$.

Both facts come from one property: x_k minimizes $\|x - x^*\|_A$ over $x_0 + \mathcal{K}_k(A, r_0)$, so

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \le \min_{\substack{P \in \mathcal{P}_k \\ P(0)=1}} \max_{\lambda \in \sigma(A)} |P(\lambda)|.$$

Each generalization below keeps some of these ingredients and drops others. Table 1 shows which, and it is the most useful way to read the literature.

Ingredient of linear CG	What happens when it is lost
A symmetric	No short (3-term) recurrence that is also optimal (Faber–Manteuffel theorem [11]). One must choose between GMRES (optimal, long recurrence) and BiCG/BiCGStab (short, not optimal).
A positive definite	$\|\cdot\|_A$ is not a norm. Use MINRES/SYMMLQ [29], or in optimization *detect* negative curvature $p^\mathsf{T} A p \le 0$ and stop (Steihaug–Toint).
Constant Hessian (f quadratic)	Conjugacy holds only approximately. α_k needs a line search, and the different formulas for β_k stop being equivalent. This is nonlinear CG (Section 4).
Exact arithmetic	Orthogonality is lost, so convergence is delayed, though not destroyed [25, 4].

Table 1: Each extension of CG drops one ingredient of the linear method.

2 Sense (i): other linear systems (still linear, but not SPD)

- **Least squares** $\min \|Jx - r\|^2$: apply CG to $J^\mathsf{T} J x = J^\mathsf{T} r$ without forming $J^\mathsf{T} J$ (CGLS/CGNR). LSQR and LSMR are the numerically stable versions [30, 13]. The effective condition number is $\kappa(J)^2$. This is the linear-algebra core of Gauss–Newton (U21).
- **Symmetric indefinite**: MINRES minimizes $\|r_k\|_2$ over the same Krylov space [29].
- **Nonsymmetric**: GMRES, BiCG, BiCGStab; see Saad's book [35].

These are relatives of CG, not CG itself. Optimization uses them mostly through Gauss–Newton and KKT systems.

3 Sense (ii): linear CG inside Newton and trust-region methods

This is the most important practical use of linear CG for general f. At x_k the Newton step solves

$$\nabla^2 f(x_k)\, p = -\nabla f(x_k),$$

which is exactly a quadratic problem of the U18–U19 type with $A = \nabla^2 f(x_k)$ and $b = -\nabla f(x_k)$. CG needs only the product $v \mapsto \nabla^2 f(x_k)v$, which can be computed by automatic differentiation or a finite difference of gradients, so the Hessian never has to be formed.

Inexact (truncated) Newton. Stop CG when $\left\|\nabla^2 f_k\, p + \nabla f_k\right\| \le \eta_k \left\|\nabla f_k\right\|$. Dembo, Eisenstat and Steihaug [10] proved that $\eta_k \to 0$ gives superlinear convergence and $\eta_k = O(\|\nabla f_k\|)$ gives quadratic convergence ([28, Thm. 7.2]). If CG meets $p_j^{\mathsf{T}} \nabla^2 f_k\, p_j \le 0$, the Hessian is not positive definite, and CG stops (returning $-\nabla f_k$ on the first iteration). This is N&W Algorithm 7.1 (line-search Newton–CG); see the survey by Nash [27].

Trust region (connects to U15–U17). Steihaug [39] and Toint showed that running preconditioned CG on the model $m_k(p)$ from $p = 0$ and stopping when (a) the iterate leaves the ball $\|p\| \le \Delta_k$ or (b) negative curvature appears gives a step at least as good as the Cauchy point. That is enough for the global convergence theory of U17. The first CG iterate *is* the Cauchy point, so the dogleg of U16 is a two-segment version of the CG path. GLTR [18] continues past the boundary with Lanczos.

Large-scale learning. "Hessian-free" optimization of neural networks [24] is truncated Newton–CG with Gauss–Newton curvature products.

Preconditioning. The matrix M in `linear_cg.py` plays the same role here. In practice M is incomplete Cholesky, a diagonal, an L-BFGS matrix, or a randomized low-rank (Nyström) approximation [3, 14].

4 Sense (iii): nonlinear CG for general smooth f

4.1 The algorithm

Replace r_k by $g_k = \nabla f(x_k)$, the closed-form α_k by a line search, and keep the direction recurrence:

Algorithm 1 Nonlinear CG (Fletcher–Reeves 1964 [12] and successors)

1: Given x_0; set $g_0 = \nabla f(x_0)$, $d_0 = -g_0$.
2: **for** $k = 0, 1, 2, \ldots$ until $\|g_k\| \le \varepsilon$ **do**
3: Choose α_k by a (strong) Wolfe line search along d_k ▷ agenda default: $c_1 = 10^{-4}$, $c_2 = 0.1$
4: $x_{k+1} = x_k + \alpha_k d_k, \quad g_{k+1} = \nabla f(x_{k+1}), \quad y_k = g_{k+1} - g_k$
5: $d_{k+1} = -g_{k+1} + \beta_{k+1} d_k$ (restart: $\beta_{k+1} = 0$ every n steps or when d_{k+1} is not a descent direction)
6: **end for**

Memory is a few vectors, and cost per iteration is one gradient plus the line search. This is why nonlinear CG is used when n is very large and even L-BFGS memory is too costly.

4.2 The choice of β_k

Proposition 1 (All formulas agree on quadratics)**.** *If $f(x) = \frac{1}{2} x^{\mathsf{T}} A x - b^{\mathsf{T}} x$ with $A \succ 0$ and α_k is the exact minimizer along d_k, then $\beta^{\mathrm{HS}} = \beta^{\mathrm{FR}} = \beta^{\mathrm{PRP}} = \beta^{\mathrm{DY}} = \beta^{\mathrm{HZ}}$, and Algorithm 1 coincides with linear CG* (1) *with $M = I$.*

Name	β_{k+1}	Year	Key property
Hestenes–Stiefel (HS)	$\dfrac{g_{k+1}^{\mathsf{T}} y_k}{d_k^{\mathsf{T}} y_k}$	1952 [22]	satisfies $d_{k+1}^{\mathsf{T}} y_k = 0$ (conjugacy)
Fletcher–Reeves (FR)	$\dfrac{\|g_{k+1}\|^2}{\|g_k\|^2}$	1964 [12]	global conv. with strong Wolfe, $c_2 < \frac{1}{2}$ [1]
Polak–Ribière–Polyak (PRP)	$\dfrac{g_{k+1}^{\mathsf{T}} y_k}{\|g_k\|^2}$	1969 [31, 32]	auto-restart after tiny steps; may cycle [33]
PR+	$\max\{\beta^{\mathrm{PRP}}, 0\}$	1992 [16]	global conv. (Gilbert–Nocedal)
Dai–Yuan (DY)	$\dfrac{\|g_{k+1}\|^2}{d_k^{\mathsf{T}} y_k}$	1999 [7]	descent + conv. with *standard* Wolfe
Hager–Zhang (HZ)	$\dfrac{1}{d_k^{\mathsf{T}} y_k}\Big(y_k - 2d_k \dfrac{\|y_k\|^2}{d_k^{\mathsf{T}} y_k}\Big)^{\mathsf{T}} g_{k+1}$	2005 [19]	$g_k^{\mathsf{T}} d_k \le -\frac{7}{8}\|g_k\|^2$ for any α_k

Table 2: Main choices of β. Hundreds of hybrids exist; see the surveys [20, 5, 26, 2].

Sketch. Exact line search gives $g_{k+1}^{\mathsf{T}} d_k = 0$. Linear CG gives $g_{k+1}^{\mathsf{T}} g_k = 0$ (residuals are mutually orthogonal). Hence

$$g_{k+1}^{\mathsf{T}} y_k = \|g_{k+1}\|^2, \qquad d_k^{\mathsf{T}} y_k = d_k^{\mathsf{T}} g_{k+1} - d_k^{\mathsf{T}} g_k = -(-g_k + \beta_k d_{k-1})^{\mathsf{T}} g_k = \|g_k\|^2,$$

using $d_{k-1}^{\mathsf{T}} g_k = 0$. So all numerators equal $\|g_{k+1}\|^2$ and all denominators equal $\|g_k\|^2$, which is the FR value. The second HZ term vanishes because $d_k^{\mathsf{T}} g_{k+1} = 0$. Finally $y_k = \alpha_k A d_k$, so $d_{k+1}^{\mathsf{T}} A d_k = 0$ is the HS conjugacy condition. □

So the formulas differ only through how they behave *away* from the quadratic model, with inexact line searches and a Hessian that changes from point to point. That is the whole subject of the literature in Table 2.

4.3 What is known about convergence

- **Tool:** Zoutendijk's condition $\sum_k \cos^2\theta_k \|g_k\|^2 < \infty$ under Wolfe steps (U5). Every global result proves either $\cos\theta_k$ bounded away from 0 or $\sum \|d_k\|^{-2} = \infty$.
- **FR:** $\liminf \|g_k\| = 0$ with strong Wolfe, $c_2 < 1/2$ [1]. In practice FR can get stuck making tiny steps with $\cos\theta_k \approx 0$ (N&W §5.2), so it needs restarts.
- **PRP:** Powell [33] built a nonconvex example where PRP with *exact* line search cycles without approaching a stationary point. The fix $\beta = \max\{\beta^{\mathrm{PRP}}, 0\}$ (PR+) is globally convergent [16], and in practice PRP/PR+ is much better than FR.
- **Rate:** with exact line search and restarts every n steps, the rate is n-step quadratic, $\|x_{k+n} - x^*\| = O(\|x_k - x^*\|^2)$. Without restarts it is generally only linear, never superlinear (Crowder–Wolfe; N&W §5.2).
- **Recent worst-case analysis.** Das Gupta, Freund, Sun and Taylor [9] used computer-assisted performance estimation (PEP) to obtain the first non-asymptotic bound for FR and an improved one for PRP on L-smooth, μ-strongly convex f. Their adversarial examples show that in the worst case these methods are *no better than steepest descent with exact line search.* The P20 advantage ($\sqrt{\kappa}$ vs. κ) therefore does not carry over to nonlinear CG as a worst-case guarantee. It holds only asymptotically, near x^*, where f is nearly quadratic.

4.4 Modern implementations and the bridge to L-BFGS (U20)

- **CG_DESCENT** (Hager–Zhang): the HZ β plus an "approximate Wolfe" line search that is accurate near the solution [19]. HZ comes from Perry/Shanno's *memoryless BFGS*: a nonlinear CG direction is a quasi-Newton direction that uses only the last pair (s_k, y_k).
- **Limited-memory CG** [21]: on ill-conditioned problems, rounding makes successive gradients lose the orthogonality of the linear case. The method detects this and restores orthogonality with a short L-BFGS-type memory. It sits between nonlinear CG and L-BFGS, which is exactly the topic of U20.
- **CGOPT** (Dai–Kou) [6]: a CG direction chosen closest to the scaled memoryless BFGS direction. It is competitive with CG_DESCENT.
- **Preconditioning** carries over directly:

$$d_{k+1} = -M^{-1}g_{k+1} + \beta_{k+1}d_k, \qquad \beta_{k+1}^{\mathrm{PR}} = \frac{g_{k+1}^{\mathsf{T}} M^{-1} y_k}{g_k^{\mathsf{T}} M^{-1} g_k}.$$

  This is the same $y = M^{-1}r$ step as in `linear_cg.py`.

4.5 Further generalizations (research frontier)

- **Riemannian CG.** For f defined on a manifold (Stiefel, Grassmann, SPD matrices), d_k must be moved to the new tangent space by a vector transport before forming $-\operatorname{grad} f(x_{k+1}) + \beta d_k$. Global convergence depends on how the transport is chosen [34, 37, 36].
- **Hilbert spaces and nonlinear operator equations:** Daniel [8].
- **Monotone nonlinear equations** $F(x) = 0$**:** CG directions combined with hyperplane projection (many recent HZ-type papers).
- **High-performance linear CG:** pipelined and communication-avoiding variants hide the global reductions $r^{\mathsf{T}}y$ and $p^{\mathsf{T}}w$ [15]. These are relevant if you write your own C++ kernels.

5 Small experiment

Setup. Chained Rosenbrock $f(x) = \sum_{i=1}^{n-1} 100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2$ with $n = 100$ and $x_0 = (-1.2, 1, -1.2, 1, \dots)$. Strong Wolfe line search with $c_1 = 10^{-4}$, $c_2 = 0.1$ (the agenda default for nonlinear CG). Stop at $\|\nabla f\| \le 10^{-6}$ or after 5000 iterations. Newton–CG and L-BFGS use the SciPy implementations, which have their own line searches, so their evaluation counts are only indicative.

Method	Iterations	$f/\nabla f$ evals	Comment
Steepest descent	> 5000	53 792	not converged ($\|\nabla f\| \approx 3.0$)
FR	1226	14 431	long plateau
FR, restart every n	983	11 684	restarts help FR
DY	1476	17 712	
HS	872	9 325	
HZ	906	9 520	
PR+	**761**	**8 160**	best nonlinear CG here
L-BFGS ($m = 5$)	530	631	SciPy line search
Newton–CG	228	261 (+1869 Hv)	linear CG as inner solver

Table 3: Results for Figure 1.

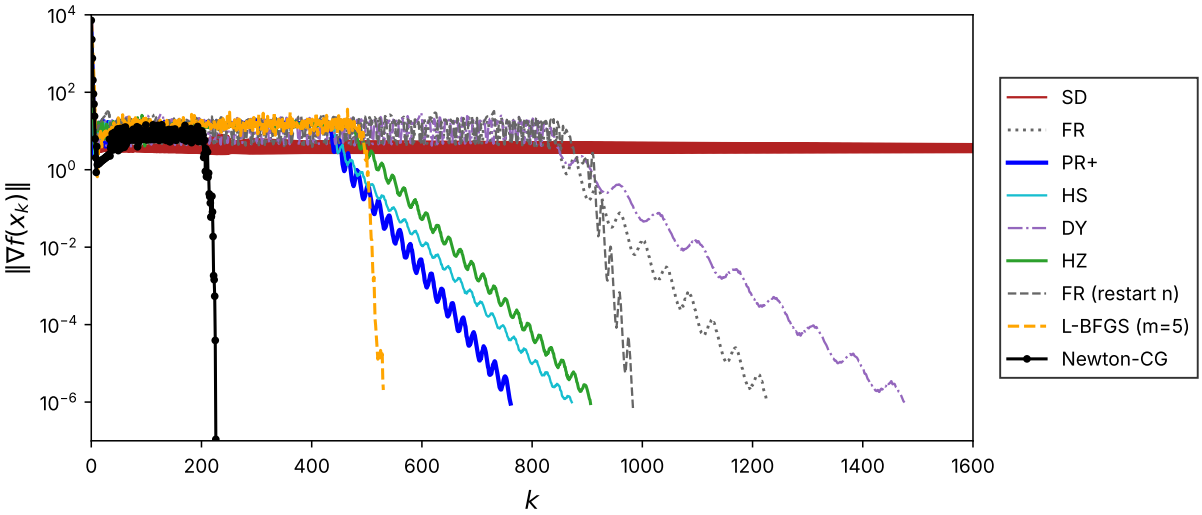


Figure 1: Gradient norm on chained Rosenbrock ($n = 100$). Every nonlinear CG variant converges; steepest descent stalls at $\|\nabla f\| \approx 3$ after 5000 iterations.

Observations. (1) The qualitative lesson of P18/P20 survives: CG-type directions beat steepest descent by orders of magnitude even on a nonconvex function. (2) The equivalence of the β formulas breaks down: FR and DY are clearly slower than PR+/HS/HZ, as the theory predicts. (3) With more memory (L-BFGS) or curvature information (Newton–CG), iteration counts drop further. This is the trade-off that U20 studies.

6 Annotated reading list

Marked ★ = suggested starting points.

History and foundations.

- ★ Hestenes & Stiefel (1952) [22]: the original paper. It already contains the HS formula and the Krylov optimality.
- Golub & O'Leary (1989) [17]: history of CG and Lanczos, 1948–1976.
- ★ Shewchuk (1994) [38]: "CG without the agonizing pain", a geometric explanation with pictures like P18.

Linear CG: theory and finite precision (continues U19).

- ★ Carson, Liesen & Strakoš (2024) [4]: CG and GMRES through computed examples. Explains why the κ-bound of P20 can be pessimistic (eigenvalue distribution matters) and how rounding delays convergence.
- Meurant & Strakoš (2006) [25]: Lanczos and CG in finite precision (Acta Numerica).
- Liesen & Strakoš (2013) [23]: book-length treatment of Krylov methods.

Preconditioning and computation.

- Benzi (2002) [3]: survey of preconditioners. Relevant to the M in `linear_cg.py`.

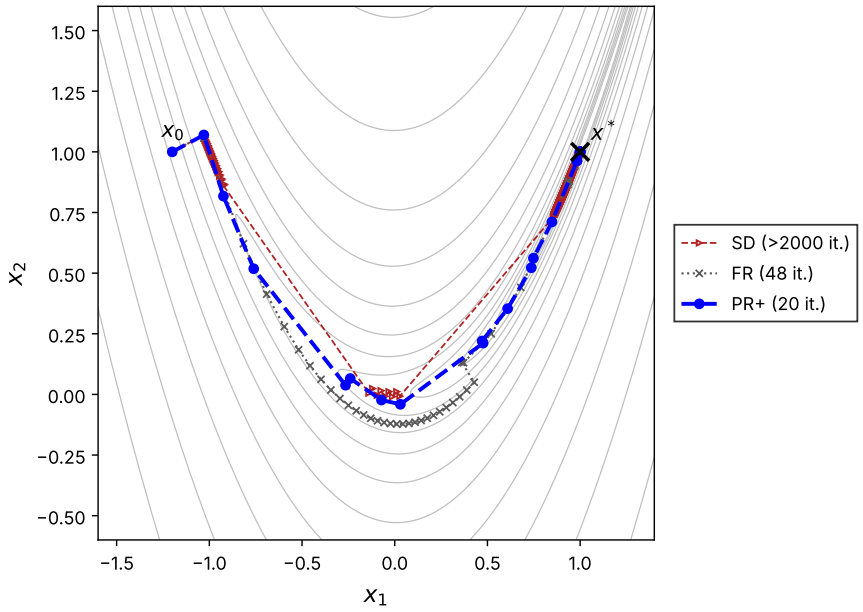


Figure 2: Same comparison in $n = 2$, drawn like P18. PR+ follows the curved valley in 20 iterations. FR takes many short steps (48 iterations). Steepest descent zig-zags and has not converged after 2000.

- Frangella, Tropp & Udell (2023) [14]: randomized Nyström preconditioner. The preconditioned condition number becomes constant once the rank is about the effective dimension.
- Ghysels & Vanroose (2014) [15]: pipelined CG for parallel machines.

Nonlinear CG (prepares U20).

- ★ Hager & Zhang (2006) [20]: *the* survey of nonlinear CG, organized around global convergence.
- Fletcher & Reeves (1964) [12]; Polak & Ribière (1969) [31]; Polyak (1969) [32]: the founding papers.
- Al-Baali (1985) [1]; Powell (1984) [33]; Gilbert & Nocedal (1992) [16]: the classical convergence results and counterexamples (N&W §5.2 is based on them).
- Dai & Yuan (1999) [7]; Hager & Zhang (2005) [19]; Dai & Kou (2013) [6]; Hager & Zhang (2013) [21]: modern methods and software.
- Dai (2011) [5]; Narushima & Yabe (2014) [26]; Andrei (2020, book) [2]: further surveys.
- ★ Das Gupta, Freund, Sun & Taylor (2025) [9]: worst-case rates of FR/PRP by computer-assisted proofs. Code is public.

CG inside second-order methods (links U7, U15–U17, U21).

- ★ Steihaug (1983) [39]: CG with trust regions.
- Dembo, Eisenstat & Steihaug (1982) [10]: inexact Newton methods.
- Nash (2000) [27]: survey of truncated Newton methods.
- Gould, Lucidi, Roma & Toint (1999) [18]: GLTR (Lanczos trust region).
- Martens (2010) [24]: Hessian-free deep learning.

Beyond $\mathbb{R}^n$.

- Ring & Wirth (2012) [34]; Sato & Iwai (2015) [37]; Sato (2022) [36]: Riemannian CG.

7 Conclusion

CG generalizes in three ways, and they should be kept separate when reading articles:

1. *Linear CG on the local quadratic model* (Newton–CG, Steihaug–Toint). All the U18–U19 theory applies to each inner solve. This is the most robust way to use CG for general f.

2. *Nonlinear CG*, the same recurrence applied directly to f. It is cheap and needs little memory, but finite termination is lost and the various β's behave differently. The research questions are global convergence with inexact line searches, sufficient descent, and restarts. The worst-case rate is no better than steepest descent's; the practical advantage is real but asymptotic.

3. *Krylov relatives* for non-SPD linear systems, and CG on manifolds.

References

[1] M. Al-Baali. Descent property and global convergence of the Fletcher–Reeves method with inexact line search. *IMA J. Numer. Anal.*, 5(1):121–124, 1985.

[2] N. Andrei. *Nonlinear Conjugate Gradient Methods for Unconstrained Optimization*. Springer, 2020.

[3] M. Benzi. Preconditioning techniques for large linear systems: a survey. *J. Comput. Phys.*, 182(2):418–477, 2002.

[4] E. Carson, J. Liesen, Z. Strakoš. Towards understanding CG and GMRES through examples. *Linear Algebra Appl.*, 2024. arXiv:2211.00953.

[5] Y.-H. Dai. Nonlinear conjugate gradient methods. In *Wiley Encyclopedia of Operations Research and Management Science*, 2011.

[6] Y.-H. Dai, C.-X. Kou. A nonlinear conjugate gradient algorithm with an optimal property and an improved Wolfe line search. *SIAM J. Optim.*, 23(1):296–320, 2013.

[7] Y.-H. Dai, Y. Yuan. A nonlinear conjugate gradient method with a strong global convergence property. *SIAM J. Optim.*, 10(1):177–182, 1999.

[8] J. W. Daniel. The conjugate gradient method for linear and nonlinear operator equations. *SIAM J. Numer. Anal.*, 4:10–26, 1967.

[9] S. Das Gupta, R. M. Freund, X. A. Sun, A. Taylor. Nonlinear conjugate gradient methods: worst-case convergence rates via computer-assisted analyses. *Math. Program.*, 213:1–49, 2025. doi:10.1007/s10107-024-02127-7.

[10] R. S. Dembo, S. C. Eisenstat, T. Steihaug. Inexact Newton methods. *SIAM J. Numer. Anal.*, 19(2):400–408, 1982.

[11] V. Faber, T. Manteuffel. Necessary and sufficient conditions for the existence of a conjugate gradient method. *SIAM J. Numer. Anal.*, 21(2):352–362, 1984.

[12] R. Fletcher, C. M. Reeves. Function minimization by conjugate gradients. *The Computer Journal*, 7(2):149–154, 1964.

[13] D. C.-L. Fong, M. Saunders. LSMR: an iterative algorithm for sparse least-squares problems. *SIAM J. Sci. Comput.*, 33(5):2950–2971, 2011.

[14] Z. Frangella, J. A. Tropp, M. Udell. Randomized Nyström preconditioning. *SIAM J. Matrix Anal. Appl.*, 44(2):718–752, 2023. doi:10.1137/21M1466244.

[15] P. Ghysels, W. Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224–238, 2014.

[16] J. C. Gilbert, J. Nocedal. Global convergence properties of conjugate gradient methods for optimization. *SIAM J. Optim.*, 2(1):21–42, 1992.

[17] G. H. Golub, D. P. O'Leary. Some history of the conjugate gradient and Lanczos algorithms: 1948–1976. *SIAM Review*, 31(1):50–102, 1989.

[18] N. I. M. Gould, S. Lucidi, M. Roma, Ph. L. Toint. Solving the trust-region subproblem using the Lanczos method. *SIAM J. Optim.*, 9(2):504–525, 1999.

[19] W. W. Hager, H. Zhang. A new conjugate gradient method with guaranteed descent and an efficient line search. *SIAM J. Optim.*, 16(1):170–192, 2005.

[20] W. W. Hager, H. Zhang. A survey of nonlinear conjugate gradient methods. *Pacific J. Optim.*, 2(1):35–58, 2006.

[21] W. W. Hager, H. Zhang. The limited memory conjugate gradient method. *SIAM J. Optim.*, 23(4):2150–2168, 2013. doi:10.1137/120898097.

[22] M. R. Hestenes, E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Nat. Bur. Standards*, 49(6):409–436, 1952.

[23] J. Liesen, Z. Strakoš. *Krylov Subspace Methods: Principles and Analysis*. Oxford University Press, 2013.

[24] J. Martens. Deep learning via Hessian-free optimization. In *Proc. ICML*, 2010.

[25] G. Meurant, Z. Strakoš. The Lanczos and conjugate gradient algorithms in finite precision arithmetic. *Acta Numerica*, 15:471–542, 2006.

[26] Y. Narushima, H. Yabe. A survey of sufficient descent conjugate gradient methods for unconstrained optimization. *SUT J. Math.*, 50(2):167–203, 2014.

[27] S. G. Nash. A survey of truncated-Newton methods. *J. Comput. Appl. Math.*, 124:45–59, 2000.

[28] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006. (Ch. 5 and §7.1.)

[29] C. C. Paige, M. A. Saunders. Solution of sparse indefinite systems of linear equations. *SIAM J. Numer. Anal.*, 12(4):617–629, 1975.

[30] C. C. Paige, M. A. Saunders. LSQR: an algorithm for sparse linear equations and sparse least squares. *ACM Trans. Math. Softw.*, 8(1):43–71, 1982.

[31] E. Polak, G. Ribière. Note sur la convergence de méthodes de directions conjuguées. *Rev. Française Informat. Recherche Opérationnelle*, 3(16):35–43, 1969.

[32] B. T. Polyak. The conjugate gradient method in extremal problems. *USSR Comput. Math. Math. Phys.*, 9(4):94–112, 1969.

[33] M. J. D. Powell. Nonconvex minimization calculations and the conjugate gradient method. In *Numerical Analysis (Dundee 1983)*, LNM 1066, Springer, 122–141, 1984.

[34] W. Ring, B. Wirth. Optimization methods on Riemannian manifolds and their application to shape space. *SIAM J. Optim.*, 22(2):596–627, 2012.

[35] Y. Saad. *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.

[36] H. Sato. Riemannian conjugate gradient methods: general framework and specific algorithms with convergence analyses. *SIAM J. Optim.*, 32(4), 2022. arXiv:2112.02572.

[37] H. Sato, T. Iwai. A new, globally convergent Riemannian conjugate gradient method. *Optimization*, 64(4):1011–1031, 2015.

[38] J. R. Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Tech. report, Carnegie Mellon University, 1994.

[39] T. Steihaug. The conjugate gradient method and trust regions in large scale optimization. *SIAM J. Numer. Anal.*, 20(3):626–637, 1983. doi:10.1137/0720042.